NEURAL FORGE / MEMORY HIVE

NF

Memory Hive HyperRecall + TokenFS

A deep implementation plan for faster swarm recall: token-addressed memory, citation-preserving bundles, and local-first indexed context without deleting source truth.

1,087	521K	FTS5	<750ms
LIVE MARKDOWN FILES	ROUGH NAIVE TOKENS	AVAILABLE LOCALLY	TARGET BUNDLE PATH

artifact-first plan

source → chunk → code → bundle → expand

> **Design principle:** source files remain truth. TokenFS is an accelerator. Every accelerated answer must cite the original path and line span.

Neural Forge operating brief: make Memory Hive recall faster as the swarm grows, without losing provenance, context, or operator trust.

Executive read

Memory Hive is already crossing the point where a file-backed memory system can no longer be treated as "just read the markdown." The live hive baseline shows:

- 1,087 markdown files
- 2,084,524 bytes of markdown
- ~521,131 rough tokens if read naively
- `memory-hive stats`: ~12.7s
- `memory-hive digest --week`: ~13.5s
- broad `memory-hive query memory`: ~12.0s and ~86.9KB stdout
- SQLite FTS5 is available locally

The right move is not to replace Memory Hive's source files. The markdown files are the legal record and the collaboration substrate. The speed layer should sit beside them: a local-first indexed token filesystem that turns growing files into stable, citation-preserving chunks, short deterministic codes, fast retrieval plans, and bounded prompt bundles.

Working name: HyperRecall / TokenFS.

Core principle:

Source files remain truth. TokenFS is an accelerator. Every accelerated answer must cite the original path + line span and be expandable back to the source.

What we are building

HyperRecall is a recall engine inside Memory Hive with five responsibilities:

1. Index every hive file incrementally.
2. Convert markdown into stable chunk records with source spans.
3. Generate compact token codes for chunks, skills, memories, agents, topics, dates, and trust level.
4. Plan recall bundles from the current prompt instead of dumping broad files.
5. Preserve context by returning citations, snippets, salience reasons, and expansion handles.

This creates a fast path for agents:

Current path: read index.md + registry + knowledge + distilled + silo files + broad search output

Target path: read compact boot capsule + ask HyperRecall for a task-shaped bundle + expand only cited source spans when needed

Why this matters now

The current approach is honest but scales linearly. Every new agent, raw learning, log, skill reference, and project note adds more bytes to scan. The user experience degrades in two ways:

- Time: command latency grows into multi-second or tens-of-second recall.
- Tokens: agents either over-read and waste context or under-read and miss important prior work.

Naive summarization is not enough. It risks deleting context, hiding contradictions, and breaking auditability. The better pattern is layered recall:

- Lossless source layer: markdown stays where it is.
- Structural index layer: manifest, chunks, citations, hashes.
- Symbolic token layer: short codes that compress references and make bundles cheap.
- Semantic/ranking layer: FTS5 first, embeddings optional later.
- Bundle layer: budget-aware prompt capsules with provenance.

Design goals

Hard requirements:

- No context loss by default.
- Every snippet cites `path:line_start-line_end`.
- Incremental rebuilds skip unchanged files.
- Works offline/local-first.
- No external vector DB required for v1.
- Does not mutate source markdown except through existing Memory Hive write paths.
- Respects lane-keeping: agents still write only to allowed silos/raw learnings.
- Keeps command output bounded by token budget.
- Detects stale citations when source file hashes change.
- Has benchmark proof before/after.

Speed targets:

- `memory-hive hyper status`: <200ms on normal hives.
- Incremental index after one file changes: <300ms.
- Common recall query: <500ms for top 20 cited chunks.
- Bundle generation: <750ms for 4K-12K token budgets.
- Full rebuild on current live hive: <10s v1, <3s later with optimized parser/cache.

Quality targets:

- Better than grep for task prompts with multiple terms.
- Never worse than current `query --paths-only` for exact terms.

- Explicitly shows why each result was selected.
- Lets the agent expand from compact code to original source.

Mental model: TokenFS

TokenFS is not a new disk filesystem. It is a token-addressed view of Memory Hive.

A file path like:

`hive/agents/hermes/memory.md`

is projected into records like:

`hc:v1.agent.hermes.memory.20260506.knox-relay.8f3a.c2`

The code is short enough to fit into prompts, deterministic enough to cache, and reversible through the local index.

A code does not replace text. It is a pointer with metadata:

- source path
- line range
- byte offsets
- file hash
- chunk hash
- heading context
- agent/kind/date/topic fields
- trust/salience score
- preview snippet

The agent can say: "I need `hc:v1.agent.hermes.memory...` expanded" and the tool reads only that span from the source file.

Proposed command surface

Add commands without disrupting current workflows:

```
memory-hive hyper init
memory-hive hyper rebuild [--full] [--dry-run]
memory-hive hyper update [--since-checkpoint <name>] [--watch]
memory-hive hyper status
memory-hive hyper query "<prompt>" [--for hermes] [--max-tokens 4000] [--json]
memory-hive hyper bundle --for hermes --task "<brief>" --max-tokens 8000 [--out file]
memory-hive hyper expand <code|path:lines> [--max-lines N]
memory-hive hyper bench [--before-after] [--json]
memory-hive hyper doctor
```

Add aliases later if it proves stable:

```
memory-hive recall ...
memory-hive bundle --smart ...
```

Keep `query`, `digest`, and `bundle` intact. HyperRecall is an opt-in fast path until it has enough tests.

Storage layout

Inside the installed hive, add a generated index area:

```
hive/.hyper/
  hyper.db              # SQLite WAL database
  manifest.json         # human-inspectable summary + schema version
  locks/                # lock files for safe concurrent rebuilds
  cache/
    bundles/            # prompt bundle cache by query hash + budget
    snippets/           # optional pre-rendered snippets
  bench/
    latest.json
    history.jsonl
```

Alternative if we want to keep generated state outside the hive repo:

```
~/.memory-hive/cache/hyper/
```

Recommendation: put `hyper.db` under `~/.memory-hive/cache/hyper/` by default and symlink/status-reference it from `hive/.hyper/` only if needed. Generated indexes should not become human-authored shared hive content.

SQLite schema v1

Use SQLite because it is local, inspectable, fast, and already available.

Tables:

```
files(
  file_id integer primary key,
  relpath text unique not null,
  surface text not null,          -- agents, learnings, knowledge, registry, tasks, curator
  kind text not null,             -- log, memory, context, raw, distilled, knowledge, registry
  agent text,
  mtime_ns integer not null,
  size_bytes integer not null,
  sha256 text not null,
  parse_status text not null,
  updated_at text not null
);

chunks(
  chunk_id integer primary key,
  file_id integer not null references files(file_id),
  stable_id text not null,
  code text unique not null,
  kind text not null,
  agent text,
  heading_path text,
  line_start integer not null,
  line_end integer not null,
  byte_start integer not null,
  byte_end integer not null,
  text text not null,
  summary text,
  token_est integer not null,
  sha256 text not null,
  simhash integer,
  created_at text not null,
  updated_at text not null
);

chunk_fts using fts5(
  text,
  summary,
  heading_path,
  relpath,
  agent,
  kind,
  content='chunks',
  content_rowid='chunk_id'
);

tokens(
  token text not null,
  chunk_id integer not null,
  weight real not null default 1.0,
  primary key(token, chunk_id)
);

entities(
  entity text not null,
  entity_type text not null,
  chunk_id integer not null,
  confidence real not null,
  primary key(entity, entity_type, chunk_id)
);

access_log(
  ts text not null,
  query_hash text not null,
  chunk_id integer not null,
  action text not null,           -- selected, expanded, dismissed, cited
```

```
  agent text,
  task_kind text
);

bundle_cache(
  key text primary key,
  created_at text not null,
  max_tokens integer not null,
  query text not null,
  agent text,
  source_fingerprint text not null,
  markdown text not null
);
```

Indexes:

```
create index idx_files_relpath on files(relpath);
create index idx_files_mtime on files(mtime_ns);
create index idx_chunks_kind_agent on chunks(kind, agent);
create index idx_chunks_code on chunks(code);
create index idx_tokens_token on tokens(token);
create index idx_entities_entity on entities(entity, entity_type);
```

Chunking strategy

Chunk markdown into stable records, not arbitrary fixed-size slices.

Priority boundaries:

1. YAML frontmatter.

2. H1/H2/H3 sections.

3. Individual log lines beginning with dates.

4. Bullets under headings.

5. Paragraph blocks.

6. Code fences as standalone chunks.

7. Tables by row group.

Each chunk stores:

- exact line range
- byte offsets
- heading ancestry
- raw text
- rough token estimate
- source hash
- chunk hash

Stability rule:

A chunk stable ID should be based on path + nearest heading + first normalized 80 chars + local ordinal. That keeps IDs stable when unrelated sections change.

Token code design

Suggested code format:

```
hc1:&lt;surface&gt;&lt;kind&gt;:&lt;agent&gt;:&lt;date&gt;:&lt;topic&gt;:&lt;sig&gt;:&lt;chk&gt;
```

Examples:

- `hc1:am:hermes:260506:knox-relay:8f3a:c2`
- `hc1:lr:hermes:260506:lica:91bc:f9`
- `hc1:kd:global:000000:human-context:43ad:11`

Legend:

- `hc1`: HiveCode schema v1
- `a/l/k/r/t/c`: surface: agents, learnings, knowledge, registry, tasks, curator
- `m/l/c/r/d/k`: kind: memory, log, context, raw, distilled, knowledge
- `agent`: agent id or global
- `date`: YYMMDD from frontmatter, log line, mtime, or 000000
- `topic`: top normalized topic slug
- `sig`: 16-bit or 24-bit SimHash/minhash prefix for similarity
- `chk`: check bits from chunk hash/file hash to detect drift

This gives agents a compact symbolic handle while preserving source expansion.

Query planner

Given the user's current task, the planner extracts:

- entities: project names, repo paths, domains, people, agent ids
- verbs: build, debug, deploy, research, continue, update, summarize, patch
- surfaces: skill, memory, log, raw learning, distilled learning, knowledge
- time hints: today, yesterday, last week, dates
- priority hints: "continue", "resume", "ship it", "all of the above"
- risk hints: destructive, credentials, gateway lifecycle, Knox relay

Ranking formula v1:

```
score =
  0.35 * bm25
+ 0.20 * exact_entity_match
+ 0.15 * recency_decay
+ 0.10 * source_trust
+ 0.10 * agent_relevance
+ 0.05 * access_frequency
+ 0.05 * kind_prior
- 0.20 * stale_hash_penalty
- 0.10 * raw_low_confidence_penalty
```

Kind priors change by task:

- Coding task: skills + repo notes + distilled + recent agent memory.
- Continuation: recent logs + active tasks + session state + agent context.
- Public copy: user voice + content skills + prior accepted copy.
- Creative task: brand assets + visual QA learnings + creative skills.
- Memory Hive ops: memory-hive skill + distilled patterns + latest raw lessons.

Bundle format

HyperRecall returns a compact bundle, not a wall of text.

```
# Memory Hive HyperRecall Bundle
query: "speed up memory hive recall"
agent: hermes
budget: 8000 tokens
index: hyper.db schema 1
source_fingerprint: 2026-05-06T08:xxZ sha256:...

## Required boot capsule
- hc1:kd:global:000000:index:... — hive/index.md:1-80
- hc1:am:hermes:260506:context:... — agents/hermes/context.md:1-90

## Top citations
1. hc1:am:hermes:260506:hyperrecall:...
   score: 0.91
   reason: exact project + task class + recent
   source: agents/hermes/memory.md:42-61
   snippet: ...

2. ...

## Expansion handles
Use `memory-hive hyper expand <code>` to read exact source spans.
```

The bundle is designed for agents and humans. It shows what was selected and why.

Skill recall acceleration

Hermes skills also grow. The same HyperRecall pattern should index skills separately.

Surfaces:

- `~/.hermes/skills/**/SKILL.md`
- skill `references/`
- skill metadata: name, description, tags, related_skills, required commands/env
- usage history: which skills were loaded for which task classes

Commands:

```
memory-hive hyper skills rebuild
memory-hive hyper skills query "discord gateway routing" --max 8
memory-hive hyper skills bundle "design PDF like neural-forge" --max-tokens 4000
```

Integration target:

Before a response, Hermes can load a tiny skill routing capsule:

```
Relevant skills for this prompt:
- memory-hive (0.96): modifying Memory Hive recall system
- memory-and-rag-systems (0.91): citation-preserving recall engine
- claude-design (0.74): PDF/HTML design artifact
```

Then the agent loads only the relevant full skills. This reduces prompt bloat while preserving mandatory skill discipline.

Memory recall acceleration

Hermes's durable memory block is currently compact but can still become crowded. HyperRecall should split memory into:

- always-injected operator profile: tiny, user-facing preferences
- always-injected safety/operational constraints: tiny, high-risk rules
- task-triggered memory: loaded via HyperRecall based on prompt
- dormant memory: searchable but not injected

Memory should be addressable by code too:

`mem1:user:voice:travis-operating-copy:4a9c`

This lets the system carry symbolic memory pointers in compact contexts and expand when needed.

Context preservation strategy

Do not choose between speed and context. Use tiered context:

Tier 0 — boot invariants:

- agent id
- hive path
- lane-keeping rules
- core user preferences

- critical gateway/Knox safety facts

Tier 1 — task capsule:

- 10-30 top citations
- snippets only
- reasons and scores
- expansion handles

Tier 2 — source expansion:

- exact line spans requested by the agent
- larger local neighborhood around a citation

Tier 3 — full source:

- entire file only when genuinely required

Tier 4 — historical session search:

- invoked only when the hive index cannot answer or when the prompt references prior conversations not written into hive

---

Agent workflow after HyperRecall

New operational preflight:

1. Read tiny boot contract.
2. Run `memory-hive hyper bundle --for <agent> --task "<user brief>" --max-tokens 6000`.
3. Load cited skills by name.
4. Expand any citation whose snippet is insufficient.
5. Act.
6. Write task-end log/raw learning.
7. Run `memory-hive hyper update` or let watcher update.

This preserves the visible Memory Hive IO Travis wants while avoiding broad reads.

Watcher and incremental updates

Implement an update mechanism that does not require full rebuilds:

- Use file mtimes + sha256 manifest for correctness.
- On macOS, use `fswatch` if installed.
- On Linux, use inotify if available.
- Fallback: polling every 5 seconds.
- Every update runs inside a lock.
- Changed files reparse and upsert chunks.

- Deleted files mark chunks stale before deletion so old bundle caches can be invalidated.

Concurrency rule:

Only one writer updates `hyper.db`. Multiple readers can query concurrently under WAL mode.

SQLite pragmas:

```
PRAGMA journal_mode=WAL;
PRAGMA synchronous=NORMAL;
PRAGMA temp_store=MEMORY;
PRAGMA mmap_size=268435456;
```

Safety and privacy

HyperRecall must not make secrets easier to leak.

Controls:

- Secret scanner before bundle output.
- Redact high-entropy tokens by default.
- Never emit `.env`, auth files, or known credential paths.
- Store generated index outside git-tracked hive content by default.
- Add `--allow-secrets` only for local explicit operator debugging.
- Keep source citations even for redacted snippets so the operator can inspect locally.

Evaluation harness

Build a fixture with live-hive queries and expected citations.

Test classes:

1. Exact path/entity recall.
2. Continuation prompts.
3. Debugging prompts.
4. Creative brand prompts.
5. Memory Hive ops prompts.
6. Skill-routing prompts.
7. Contradiction detection.
8. Secret redaction.
9. Token budget enforcement.
10. Stale index detection.

Metrics:

- latency p50/p95

- output bytes/tokens
- recall@k against expected source files
- citation validity
- expansion correctness
- bundle budget adherence
- zero secret leakage

Acceptance gate for v1:

- 80%+ recall@10 on curated fixtures
- 100% citation validity
- 100% token budget adherence
- 0 unredacted secret-like strings in default bundles
- >10x faster than current broad grep on common queries

Implementation roadmap

Phase 0 — spec and baseline

Deliverables:

- This plan.
- Skill update pointing Memory Hive agents to HyperRecall/TokenFS pattern.
- Benchmark script capturing current stats/query/digest latencies.
- Fixture list of 20 representative operator prompts.

Exit criteria:

- Baseline stored.
- Schema agreed.
- No source markdown changes required.

Phase 1 — manifest + chunker

Deliverables:

- `scripts/hyper_recall.py` or `memory-hive hyper rebuild` backend.
- Manifest generation for hive files.
- Markdown chunker with line spans and hashes.
- SQLite database with `files`, `chunks`, FTS5.
- `hyper status`, `hyper rebuild`, `hyper expand`.

Exit criteria:

- Full rebuild indexes current 1,087 markdown files.

- Every chunk expands to exact source lines.
- Rebuild is deterministic for unchanged files.

Phase 2 — lexical retrieval + bundles

Deliverables:

- FTS5/BM25 query.
- Token/entity extraction.
- Ranking formula v1.
- `hyper query` JSON + human output.
- `hyper bundle` markdown output with reasons/citations.
- Token budget limiter.

Exit criteria:

- Common query under 500ms after index built.
- Bundle under 750ms.
- Query output is smaller and more relevant than grep.

Phase 3 — skill index

Deliverables:

- Index `~/.hermes/skills` metadata and references.
- `hyper skills query`.
- Skill routing capsule generator.
- Optional integration note for Hermes skill preflight.

Exit criteria:

- Prompt mentioning Memory Hive recall ranks `memory-hive` and `memory-and-rag-systems` top 3.
- Design/PDF prompt ranks `claude-design` top 5.

Phase 4 — watcher + cache

Deliverables:

- `hyper update` incremental path.
- `hyper watch` daemon mode.
- Bundle cache with source fingerprint invalidation.
- Locking and concurrent reader tests.

Exit criteria:

- Updating one changed file does not rescan all files.
- Bundle cache invalidates when cited source changes.

Phase 5 — semantic layer optional

Deliverables:

- Optional local embeddings behind feature flag.
- No external dependency required.
- Hybrid lexical + vector ranking.
- Dedupe/near-duplicate clustering via SimHash/MinHash.

Exit criteria:

- Semantic layer improves recall fixtures without increasing default latency too much.
- Lexical-only mode remains fully supported.

Phase 6 — Hermes integration

Deliverables:

- Minimal `memory_hive_recall` Hermes tool or CLI wrapper.
- Prompt preflight pattern update.
- Context compression uses codes/citations instead of opaque summaries.
- Post-task ritual triggers incremental index update.

Exit criteria:

- Agents visibly run HyperRecall bundle instead of broad reads for non-trivial prompts.
- Expanded citations prove context is recoverable.

Creative outside-the-box extensions

1. Context checksum handoffs

Every agent handoff includes:

- bundle hash
- source fingerprint
- top citation codes
- expanded citations used

This makes swarm work auditable. Another agent can reconstruct the context that shaped the decision.

2. Memory heat map

Use access logs to show which memories are actually used. Hot memories stay compact and injected. Cold memories move to searchable dormant storage. This prevents memory bloat without deleting knowledge.

3. Contradiction radar

When new raw learning conflicts with distilled knowledge, HyperRecall flags it:

`new chunk similar to old chunk, opposite action words, same entity`

Example: one note says restart gateway freely; newer note says avoid gateway restarts. The curator gets a conflict card.

4. Swarm context packets

For multi-agent jobs, produce role-specific bundles:

- researcher packet
- builder packet
- reviewer packet
- visual QA packet

Same source truth, different ranked slices. Less duplicate token burn across swarm agents.

5. Code-as-memory anchors

For repos, bind memory chunks to code paths:

- `memory mentions Pantheon relay`
- source code path exists
- current git hash

If code changes, related memories get a stale warning.

6. Recall simulation mode

Before a task, run:

```
memory-hive hyper simulate "continue pantheon knox relay work"
```

It shows what context would be injected and what would be omitted. This lets Travis audit recall quality before trusting automation.

7. Compression with reversible handles

Instead of `summary: Knox relay works`, inject:

`Knox relay ops: hc1:am:hermes:260505:knox-relay:8f3a:c2 + hc1:sd:global:260504:relay-constraints:91bc:f9`

The sentence is tiny, but the source can be expanded exactly.

Risks

Risk: index drift. Mitigation: file hashes, source fingerprint, stale flags, doctor command.

Risk: lost context through ranking. Mitigation: citations, expansion handles, fallback broad search, recall fixtures.

Risk: false confidence from summaries. Mitigation: summaries marked derived; snippets always cite source; source remains truth.

Risk: generated DB gets committed. Mitigation: keep under cache or add `.gitignore` entries.

Risk: command surface gets too complex. Mitigation: keep old commands; add `hyper` namespace; promote only after stable.

Risk: token codes become opaque. Mitigation: human-readable fields in code and `hyper explain <code>`.

First engineering cut

Implement as one Python-backed subcommand called by the shell CLI. Keep shell wrapper minimal.

Files:

```
scripts/hyper_recall.py
scripts/hyper_recall_schema.sql
tests/hyper/test_chunker.py
tests/hyper/test_recall.py
tests/hyper/fixtures/
```

Then patch `memory-hive` shell dispatch:

```
hyper)
  shift
  exec python3 "$INSTALL_DIR/scripts/hyper_recall.py" --install-dir "$INSTALL_DIR" "$@"
  ;;
```

This avoids turning the existing 6,600-line POSIX shell file into a database application.

Minimal v1 algorithm

Pseudo-flow:

```
def rebuild():
    files = discover_markdown(hive)
    for file in files:
        stat = file.stat()
        sha = sha256(file)
        if unchanged(file, stat, sha):
            continue
        chunks = chunk_markdown(file)
        upsert_file(file, stat, sha)
        replace_chunks(file_id, chunks)
        update_fts(file_id)


def query(prompt, agent=None, max_tokens=4000):
    features = parse_prompt(prompt)
    candidates = fts_search(features.terms)
    candidates += exact_entity_lookup(features.entities)
    ranked = rank(candidates, features, agent)
    return budgeted_bundle(ranked, max_tokens)


def expand(code):
    chunk = db.get_chunk_by_code(code)
    verify_file_hash_or_mark_stale(chunk)
    return read_source_span(chunk.relpath, chunk.line_start, chunk.line_end)
```

Operator decision

Recommended next move:

Build Phase 1 and Phase 2 as a branch in the canonical Memory Hive repo. Do not wire it into mandatory preflight yet. First prove:

- index correctness
- citation expansion
- query latency
- bundle quality
- no secret leakage

Once the fast path is verified, update the Memory Hive boot contract from "read these files" to "read boot capsule + run HyperRecall bundle + expand citations as needed."

That is the hyper-speed version: faster recall without turning memory into vibes.